

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

APLICAȚIE PENTRU LOCALIZAREA AUTOTURISMELOR (GPS)

Coordonator științific: Ș.l. Dr. Ing. Boicescu Laurențiu

Studenti: Cornilă Kevin-Andrei, Gorîn Tudor

Grupa: 434D

2025-2026

1. INTRODUCERE

1.1. Obiectivul lucrării

Scopul acestei lucrări este dezvoltarea unui sistem software pentru monitorizarea și urmărirea poziției autoturismelor, utilizând tehnologii de programare în internet. Sistemul propus permite colectarea datelor de localizare de la un dispozitiv, transmiterea acestora prin internet către un server central și prelucrarea lor pentru obținerea unor informații relevante despre deplasarea dispozitivului.

Aplicația va permite vizualizarea poziției curente, istoricul traseului parcurs, precum și calculul unor parametri precum viteza medie sau distanța parcursă. Soluția este bazată pe o arhitectură client-server, în care serverul este implementat în Java și gestionează atât stocarea datelor într-o bază de date, cât și comunicarea cu aplicațiile client.

Prin realizarea acestui proiect se urmărește aplicarea conceptelor studiate în cadrul disciplinei Tehnologii de Programare în Internet, precum dezvoltarea de aplicații, utilizarea serviciilor web și gestionarea comunicației între componente software.

Realizarea acestui proiect a fost motivată de interesul pentru dezvoltarea aplicațiilor distribuite și a sistemelor de comunicații, în special în contextul utilizării serviciilor bazate pe locație. Monitorizarea vehiculelor reprezintă o problemă reală, cu aplicații practice în domenii precum transportul, logistica sau managementul flotelor auto.

Soluția propusă valorifică o metodă flexibilă și accesibilă de colectare și transmitere a datelor. Prin utilizarea unei arhitecturi client-server și a tehnologiilor web, sistemul permite transmiterea în timp real a informațiilor și accesarea acestora de la distanță.

Astfel, aplicația contribuie la dezvoltarea unor soluții practice și scalabile, care pot fi extinse ulterior pentru utilizări reale.

1.2. Conținutul lucrării

Lucrarea este structurată în mai multe capitole. În primul capitol este prezentată introducerea și obiectivul sistemului propus. Capitolul următor descrie arhitectura generală a sistemului, evidențiind principalele componente și fluxul de date dintre acestea. Ulterior, sunt detaliate modulele aplicației, incluzând partea de achiziție a datelor, serverul de procesare și

aplicația client. De asemenea, este prezentată structura bazei de date utilizate pentru stocarea informațiilor. În final, sunt descrise aspecte legate de implementare și testare.

1.3. Contribuții



Figura 1 - Schema bloc generală

Arhitectura sistemului a fost reprezentată pe două niveluri. La nivel general, sistemul este alcătuit dintr-un modul de achiziție a datelor de localizare, un canal de comunicație prin internet, un server Java pentru procesare, o bază de date pentru stocare și o aplicație client pentru vizualizare. La nivel detaliat, fiecare componentă este împărțită în module software specializate pentru colectare, transmitere, procesare, stocare și afișare a datelor.

Principala contribuție a acestei lucrări constă în proiectarea unei arhitecturi software complete pentru un sistem de monitorizare a autoturismelor, bazat pe tehnologii web și aplicații distribuite.

Sistemul propus utilizează un telefon Android ca sursă de date de localizare, eliminând necesitatea unor module hardware dedicate, acesta ocupând un rol important pentru simularea comportamentului autoturismului. Pe viitor, implementări ulterioare pot conține un modul dedicat hardware de tip GSM pentru achiziția de date, aplicația fiind adaptată pentru această soluție.

Datele sunt transmise prin internet către un server central implementat în Java, care se ocupă de procesarea și stocarea acestora într-o bază de date relațională, de aici începând implementarea concretă a aplicației Java spre vizualizarea unor anumite funcții.

Lucrarea propune implementarea unor funcționalități precum:

- vizualizarea poziției curente a vehiculului;
- afișarea istoricului traseului;
- calculul vitezei medii;
- analiza deplasărilor în timp
- interfață pentru utilizarea ușoară a funcțiilor.

Astfel, o schemă bloc care utilizează aceste implementări spre buna funcționare a aplicației, cât și simularea scopului ei va fi:



Figura 2 - Schema bloc

Pornind de la arhitectura sistemului, formată din componentele principale telefon Android - Internet - Server Java - Bază de date - Client Java, se poate defini o arhitectură software mai detaliată, orientată spre implementarea efectivă a aplicației. Această arhitectură este organizată pe module software, fiecare corespunzând unei componente din sistem și fiind implementat folosind tehnologii specifice mediului Java și dezvoltării aplicațiilor web.

În cadrul modului de achiziție a datelor, implementat la nivelul telefonului Android, aplicația utilizează serviciile de localizare oferite de sistemul de operare pentru obținerea coordonatelor GPS. Datele colectate, precum latitudinea, longitudinea, viteza și momentul înregistrării, sunt prelucrate și organizate într-o structură de tip obiect, urmând să fie transmise către server. Transmiterea se realizează prin intermediul unui modul de comunicație care utilizează cereri HTTP către API-ul serverului.

Componenta centrală a sistemului este reprezentată de serverul implementat în Java, utilizând framework-ul Spring Boot. Acesta este organizat pe mai multe straturi, conform arhitecturii specifice aplicațiilor moderne. Primul nivel este reprezentat prin REST, definind „endpoint-urile” prin care serverul primește datele de la aplicația Android și oferă informații aplicației client. Aceste „endpoint-uri” permit atât trimiterea coordonatelor GPS, cât și interogarea datelor stocate.

Un al doilea nivel este reprezentat de layer-ul de servicii, unde este implementată logica aplicației. Acest modul se ocupă de validarea datelor primite, procesarea acestora și calculul unor parametri precum viteza medie sau istoricul traseului. Separarea acestui nivel permite o organizare clară a codului și o flexibilitate crescută în dezvoltarea ulterioară.

Interacțiunea cu baza de date este realizată prin intermediul unui layer de tip repository, utilizând tehnologii specifice precum Spring Data. Acest modul permite efectuarea operațiilor de tip inserare, actualizare și interogare asupra datelor, fără a necesita gestionarea directă a interogărilor SQL în codul principal al aplicației.

Baza de date reprezintă componenta de stocare a sistemului și este utilizată pentru memorarea pozițiilor GPS și a informațiilor asociate. Structura acesteia este proiectată astfel

încât să permită stocarea eficientă a unui număr mare de înregistrări, fiecare corespunzând unui punct de localizare.

Aplicația client, implementată în Java, utilizează API-ul expus de server pentru a obține datele necesare. Aceasta include un modul de comunicare care realizează cereri HTTP către server și un modul de interfață grafică pentru afișarea informațiilor. Funcționalitățile principale includ vizualizarea poziției curente, afișarea traseului parcurs și prezentarea unor statistici relevante.

Prin această abordare modulară și utilizarea tehnologiilor moderne precum Spring Boot și API-uri REST, sistemul este organizat într-o manieră clară și extensibilă. Fiecare componentă are un rol bine definit, iar comunicarea dintre acestea se realizează prin interfețe standardizate, ceea ce facilitează dezvoltarea și testarea aplicației.

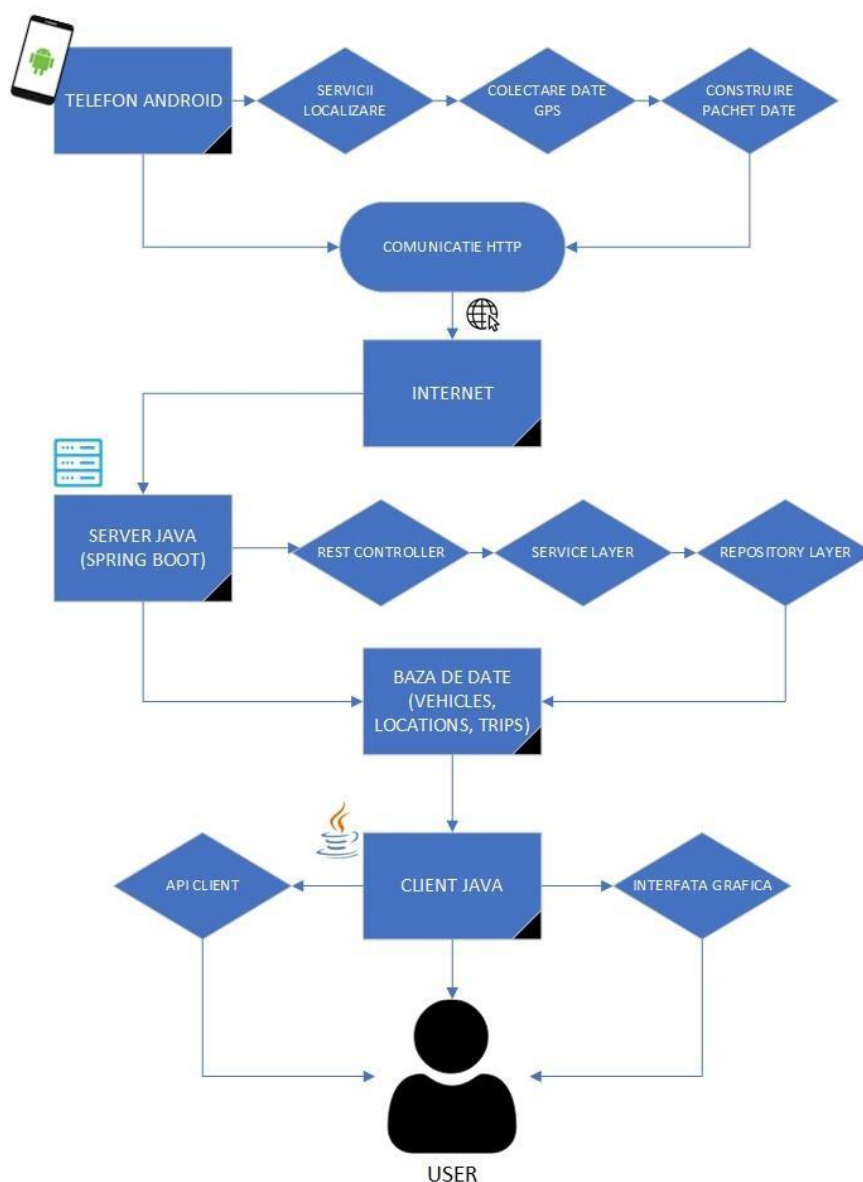


Figura 3 - Arhitectura sistemului

2. IMPLEMENTARE

2.1. Implementarea modului de achiziție de date (Android Studio)

Acest capitol detaliază prima etapă de implementare a proiectului, concentrându-se pe dezvoltarea aplicației mobile cu rol de client pentru colectarea coordonatelor geografice. Așa cum a fost definit în arhitectura sistemului, sistemul propus utilizează un telefon Android ca sursă de date de localizare, eliminând necesitatea unor module hardware dedicate, acesta ocupând un rol important pentru simularea comportamentului autoturismului. Aplicația utilizează serviciile de localizare oferite de sistemul de operare pentru obținerea coordonatelor GPS.

Proiectul a fost dezvoltat utilizând mediul integrat de dezvoltare Android Studio, având ca limbaj de programare Java. Pentru a asigura funcționalitatea aplicației, fișierul de configurare „build.gradle.kts” a fost modificat pentru a include biblioteci externe esențiale:

- Google Play Services Location (com.google.android.gms:play-services-location:21.2.0): utilizat pentru accesarea eficientă și precisă a senzorului GPS al dispozitivului prin intermediul API-ului „FusedLocationProviderClient”.
- OkHttp3 (com.squareup.okhttp3:okhttp:4.12.0): o bibliotecă HTTP performantă, necesară pentru realizarea cererilor de rețea către serverul central.
- Gson (com.google.code.gson:gson:2.10.1): utilizat pentru serializarea obiectelor Java în format JSON, facilitând astfel transmiterea datelor.

Pentru ca aplicația să poată funcționa conform scopului declarat, este necesară declararea permisiunilor corespunzătoare în fișierul „AndroidManifest.xml”. Au fost solicitate permisiunile pentru accesul la locația precisă (ACCESS_FINE_LOCATION) și la locația aproximativă (ACCESS_COARSE_LOCATION), precum și permisiunea de acces la rețeaua de internet (INTERNET). De asemenea, având în vedere că în faza de dezvoltare și testare comunicația se realizează cu un server local, a fost necesară adăugarea atributului „android:usesCleartextTraffic=<<true>>” în eticheta „<application>”.

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.INTERNET" />

<application
    android:usesCleartextTraffic="true"
    android:theme="@style/Theme.GPSCarTracker">
</application>
```

Figura 4 - cod AndroidManifest.xml

Interfața grafică a aplicației a fost menținută la un nivel simplu și intuitiv, fiind definită în fișierul „activity_main.xml”. Aceasta este compusă dintr-un layout de tip „LinearLayout” cu orientare verticală, care conține:

1. Un buton (btnSendLocation): proiectat inițial pentru declanșarea manuală a trimerii locației.
2. Un element de tip text (txtStatus): esențial pentru oferirea de feedback utilizatorului în timp real cu privire la starea permisiunilor, statusul colectării datelor sau eventualele erori de rețea.

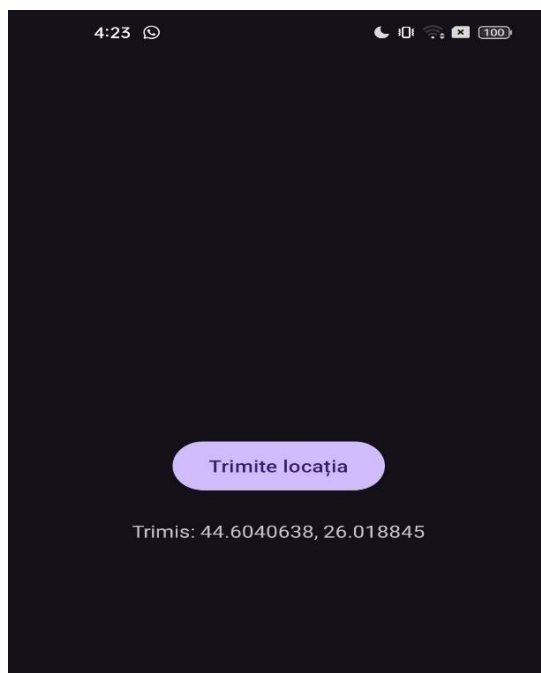


Figura 5 - Interfață aplicație Android

Datele colectate, precum latitudinea, longitudoinea, viteza și momentul înregistrării, sunt prelucrate și organizate într-o structură de tip obiect, urmând să fie transmise către server. În acest sens, a fost creată clasa „LocationData.java”, care încapsulează parametrii specifici: vehicleId (identificatorul vehiculului, hardcodat momentan "CAR_1"), latitude, longitude, speed și timestamp. Transmiterea se realizează prin intermediul unui modul de comunicație care utilizează cereri HTTP către API-ul serverului. Această funcționalitate a fost izolată în clasa „APIClient.java”. Clasa definește adresa serverului local (rețeaua locală) și folosește un client „OkHttpClient” pentru a construi și trimite o cerere de tip POST, având corpul mesajului formatat ca JSON.

```
public void sendLocation(String json, Callback callback) {  
    RequestBody body = RequestBody.create(  
        json,  
        MediaType.get("application/json; charset=utf-8")  
    );  
  
    Request request = new Request.Builder()  
        .url(SERVER_URL)  
        .post(body)  
        .build();  
  
    client.newCall(request).enqueue(callback);  
}
```

Figura 6 - cod APIClient.java

Activitatea principală (MainActivity.java) reprezintă nucleul aplicației mobile, coordonând ciclul de viață al interfeței, permisiunile și logica.

Procesul de verificare a permisiunilor: la inițializarea activității (onCreate), aplicația verifică mai întâi dacă dispune de drepturile necesare pentru a citi datele GPS. Dacă permisiunea nu este acordată, aceasta este solicitată dinamic utilizatorului prin metoda „ActivityCompat.requestPermissions”. Răspunsul utilizatorului este captat în metoda suprascrisă „onRequestPermissionsResult”, care, în caz afirmativ, declanșează procesul de localizare.

Achiziția recurentă a datelor: pentru a simula deplasarea continuă a unui vehicul, s-a implementat un mecanism de citire periodică a locației. Acest mecanism este gestionat prin intermediul unui obiect „Handler” și al unui „Runnable”, care execută metoda de preluare și trimitere a datelor la un interval regulat de 30.000 de milisecunde (30 de secunde).

```
private void startLocationUpdates() {
    txtStatus.setText("Status: tracking pornit...");

    runnable = new Runnable() {
        @Override
        public void run() {
            getLocationAndSend();
            handler.postDelayed(this, 30000); // la fiecare 30 secunde
        }
    };
    handler.post(runnable);
}
```

Figura 7 - cod MainActivity.java (delay)

Metoda „getLocationAndSend()” utilizează „FusedLocationProviderClient” pentru a obține ultima locație cunoscută a dispozitivului. În cazul în care citirea are succes, obiectul de tip Location generat de sistemul de operare Android este transformat într-un obiect de tip „LocationData”.

Serializarea și transmiterea datelor: folosind biblioteca Gson, obiectul „LocationData” este serializat într-un șir de caractere JSON. Acest JSON este trimis metodei „sendLocation” din clasa „APIClient”. Deoarece apelul de rețea este asincron și rulează pe un thread separat pentru a nu bloca interfața grafică, actualizarea elementului de tip „TextView” (txtStatus) cu răspunsul primit de la server (fie mesaj de succes cu coordonatele trimise, fie cod de eroare) se face în interiorul metodei „runOnUiThread”. Acest flux asigură o colectare robustă a datelor și transmiterea lor în timp real către serverul Java central pentru a fi procesate și stocate în baza de date.

2.2. Implementarea serverului central (Spring Boot)

Componenta centrală a sistemului este reprezentată de serverul implementat în Java, utilizând framework-ul Spring Boot. Acesta este responsabil pentru recepționarea datelor de

localizare, procesarea acestora și expunerea lor către viitoarele aplicații client. Serverul este organizat pe mai multe straturi, conform arhitecturii specifice aplicațiilor moderne, separând clar responsabilitățile.

Pentru a gestiona informațiile vehiculelor, a fost definită clasa „LocationData.java”. Aceasta acționează ca un container pentru datele recepționate, încapsulând atribute fundamentale precum: vehicleId, latitude, longitude, speed și timestamp. Această modelare permite reprezentarea fiecărui punct de pe traseu sub forma unui obiect independent în memorie. Interacțiunea și gestionarea acestor obiecte sunt realizate prin intermediul unui layer de tip repository, utilizând tehnologia Spring Data. Interfața „LocationRepository” extinde „JpaRepository”, abstractizând complet mecanismul de stocare de la nivelul logicii aplicației. Acest modul permite efectuarea operațiilor de tip inserare și interogare, definind metode personalizate pentru extragerea și ordonarea datelor pe baza identificatorului vehiculului:

- findByVehicleIdOrderByTimestampAsc: returnează o listă completă de obiecte pentru reconstrucția traseului istoric.
- findTopByVehicleIdOrderByTimestampDesc: returnează ultimul obiect înregistrat, reprezentând poziția curentă.

Un al doilea nivel este reprezentat de layer-ul de servicii, unde este implementată logica aplicației. Clasa „LocationService” preia datele de la nivelul repository și execută operații de procesare asupra acestora. Separarea acestui nivel permite o organizare clară a codului și o flexibilitate crescută în dezvoltarea ulterioară. Principalele funcționalități analitice implementate includ calculul vitezei medii: metoda „getAverageSpeed” parcurge lista punctelor de pe traseu și calculează media aritmetică a valorilor de viteză transmise de senzorul GPS; și calculul distanței totale parcurse: metoda „getTotalDistance” însumează distanțele dintre punctele GPS succesive. Pentru a asigura precizia pe suprafața sferică a Pământului, distanța dintre oricare două coordonate geografice este determinată utilizând formula matematică Haversine:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$
$$c = 2 \cdot \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$
$$d = R \cdot c$$

Figura 8 - formula Haversine (ϕ_1, ϕ_2 reprezintă latitudinile punctelor, convertite în radiani; $\Delta\phi$ și $\Delta\lambda$ reprezintă diferențele de latitudine, respectiv longitudine; R este raza medie a Pământului (aprox. 6371 km); d reprezintă distanța rezultată).

Primul nivel de interacțiune cu exteriorul este reprezentat prin REST, definind „endpoint-urile” prin care serverul primește datele de la aplicația Android și va oferi informații către clientul Java. Clasa „LocationController” este configurată la ruta „/api/location” și expune următoarele funcționalități: recepția datelor (POST /api/location), endpoint dedicat pentru primirea pachetelor JSON de la dispozitivul mobil și instanțierea acestora sub formă de obiecte „LocationData”; interogarea stării curente (GET /current/{vehicleId}), returnează locația imediată a dispozitivului; generarea traseului (GET /route/{vehicleId}), furnizează lista completă a locațiilor ordonate cronologic; statistici (GET /avgSpeed/{vehicleId} și GET /distance/{vehicleId}), apeluri ce declanșează calculele din layer-ul de servicii și returnează valorile cerute. Aceste „endpoint-uri” permit o comunicare standardizată și o decuplare eficientă între clientul care colectează datele și sistemul central de analiză.

2.3. Îmbunătățirea serverului central (Spring Boot)

Prima modificare importantă realizată la nivelul serverului a fost extinderea structurii bazei de date MySQL. În etapa precedentă, baza de date stoca doar informațiile esențiale legate de coordonatele GPS și viteza vehiculului. Pentru noua versiune a sistemului a fost necesară introducerea unui nou câmp numit „session_id”, utilizat pentru separarea sesiunilor de transmitere GPS. Structura actuală a bazei de date este următoarea:

```
CREATE DATABASE gpsdb;
USE gpsdb;

CREATE TABLE locations (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    vehicle_id VARCHAR(50),
    latitude DOUBLE,
    longitude DOUBLE,
    speed FLOAT,
    timestamp BIGINT,
    session_id BIGINT
);
```

Figura 9 - baza de date

Introducerea câmpului session_id reprezintă una dintre cele mai importante îmbunătățiri aduse sistemului. Acesta permite delimitarea logică a traseelor GPS în funcție de sesiunile de funcționare ale aplicației mobile. Astfel, dacă dispozitivul mobil transmite coordonate GPS pentru o anumită perioadă de timp, apoi aplicația sau serverul este oprit, iar ulterior transmiterea este reluată dintr-o altă locație, sistemul nu va mai uni punctele vechi cu cele noi printr-o singură linie continuă pe hartă.

Această problemă apare frecvent în sistemele simple de tracking GPS, unde traseele erau desenate exclusiv pe baza ordinii temporale a coordonatelor. Prin utilizarea unui identificator de sesiune, traseele sunt grupate separat și afișate independent, ceea ce oferă o

reprezentare mult mai realistă și profesională a deplasărilor vehiculului. La nivelul serverului Spring Boot au fost adăugate mai multe endpoint-uri REST suplimentare pentru prelucrarea și furnizarea datelor către interfața grafică. În versiunea nouă, serverul nu mai are doar rolul de a salva coordonatele GPS în baza de date, ci realizează și procesări statistice asupra acestora.

Printre funcționalitățile implementate în server se numără:

- calculul distanței totale parcurse;
- calculul vitezei medii;
- identificarea poziției vehiculului la un anumit moment temporal;
- returnarea traseelor grupate pe sesiuni;
- furnizarea poziției curente a vehiculului.

Pentru calculul distanței totale parcurse s-a implementat o logică de procesare a coordonatelor GPS folosind formula Haversine. Această formulă permite calcularea distanței dintre două puncte geografice definite prin latitudine și longitudine. Serverul parcurge toate coordonatele stocate și calculează distanța dintre fiecare două puncte consecutive aparținând aceleiași sesiuni.

În plus, pentru a evita rezultate eronate, punctele care aparțin unor sesiuni diferite nu sunt luate în calcul împreună. Astfel, sistemul nu va calcula distanțe foarte mari și nerealiste între două locații transmise la intervale mari de timp.

O altă îmbunătățire importantă adusă serverului a fost implementarea endpoint-ului pentru identificarea poziției vehiculului la un anumit timestamp. Această funcționalitate permite selectarea unei ore specifice și determinarea coordonatelor exacte la acel moment. Această caracteristică poate fi utilă în scenarii precum:

- investigarea unui incident;
- verificarea traseului într-un anumit interval orar;
- identificarea poziției unui vehicul la o anumită oră.

Pentru această funcționalitate, serverul primește un timestamp ca parametru și caută în baza de date cea mai apropiată înregistrare GPS.

De asemenea, serverul a fost optimizat pentru integrarea cu aplicația desktop Java Swing. Toate datele sunt transmise în format JSON prin intermediul API-urilor REST, iar aplicația desktop utilizează biblioteca Gson pentru conversia automată a obiectelor JSON în obiecte Java.

2.4. Realizarea interfeței grafice pentru sistemul GPS Tracker

În partea de client desktop a fost realizată o interfață grafică modernă și interactivă utilizând biblioteca Java Swing. Interfața a fost proiectată astfel încât să fie simplă, intuitivă și centrată pe informațiile importante pentru utilizator.

Aplicația desktop conține mai multe componente principale:

- dashboard live pentru monitorizare;

- hartă interactivă OpenStreetMap;
- grafic pentru analiza vitezei;
- sistem de căutare după timestamp.

Pentru comunicarea cu serverul a fost creată clasa `ApiService`, care gestionează toate cererile HTTP către serverul Spring Boot. Această clasă are rolul de a prelua:

- traseele vehiculului;
- poziția curentă;
- statisticile calculate pe server;
- poziția la un anumit timestamp.

Utilizarea unei clase separate pentru comunicația cu serverul respectă principiile arhitecturii software moderne și permite separarea logicii de rețea de logica interfeței grafice.

În ceea ce privește interfața principală, aceasta a fost realizată în clasa `MainUI`. Interfața utilizează componente Swing precum:

- `JFrame`;
- `JButton`;
- `JTextArea`;
- `JScrollPane`;
- `JPanel`.

Design-ul interfeței a fost modernizat prin utilizarea unei palete de culori personalizate și a unor butoane stilizate. Dashboard-ul principal afișează în timp real:

- poziția curentă;
- viteza instantanee;
- distanța totală;
- viteza medie;
- ultimele coordonate recepționate.

Pentru îmbunătățirea experienței utilizatorului a fost implementat și un sistem de refresh automat al datelor la fiecare 10 secunde. Astfel, aplicația actualizează permanent informațiile fără intervenția utilizatorului.

Una dintre cele mai importante funcționalități implementate este reconstrucția traseului pe hartă folosind OpenStreetMap și biblioteca Leaflet.js. Aplicația generează automat un fișier HTML care conține coordonatele traseului și îl deschide într-un browser web. Traseele sunt grupate pe sesiuni folosind `session_id`, iar fiecare sesiune este desenată independent pe hartă. Această abordare elimină problema liniilor eronate dintre două trasee separate temporal.

Pentru afișarea traseelor a fost utilizată biblioteca JavaScript Leaflet, deoarece aceasta permite integrarea rapidă a hărților OpenStreetMap și oferă suport pentru:

- markere GPS;
- polilinii;

- zoom automat;
- popup-uri informative.

În plus, aplicația permite identificarea unui punct GPS istoric direct pe hartă. Utilizatorul introduce o oră în format HH:MM, iar sistemul afișează poziția exactă a vehiculului pentru acel moment. Pentru acest mecanism a fost implementată conversia automată a timpului introdus într-un timestamp UNIX compatibil cu serverul.

Tot în partea de UI a fost implementată și o componentă pentru analiza vitezei vehiculului. Pentru aceasta a fost utilizată biblioteca JFreeChart, care generează grafice dinamice pe baza datelor GPS recepționate. Graficul permite observarea variației vitezei în timp și poate fi utilizat pentru analiza comportamentului vehiculului.

3. CONCLUZII

În cadrul acestei lucrări a fost realizată dezvoltarea unui sistem complet pentru monitorizarea și localizarea autoturismelor, utilizând tehnologii moderne de programare în Internet și o arhitectură distribuită de tip client-server. Proiectul a avut ca obiectiv principal realizarea unei soluții software capabile să colecteze date GPS în timp real, să le transmită către un server central, să le stocheze într-o bază de date și să ofere utilizatorului funcționalități de analiză și vizualizare a traseelor parcurse.

Sistemul implementat este alcătuit din mai multe componente software independente, care comunică prin intermediul unor interfețe standardizate. Componenta de achiziție a datelor este reprezentată de aplicația Android, responsabilă pentru obținerea coordonatelor GPS utilizând serviciile de localizare oferite de Google Play Services prin intermediul mecanismului FusedLocationProvider. Aplicația colectează informații precum latitudinea, longitudinea, viteza de deplasare și momentul exact al înregistrării, pe care le organizează într-o structură de tip JSON și le transmite prin HTTP către serverul central.

Aplicația Android a fost proiectată într-o manieră simplă și eficientă, fiind orientată în principal către funcționalitatea de transmitere a datelor. Interfața include un buton pentru trimiterea manuală a locației, util în etapa de testare și verificare a comunicației cu serverul, precum și un mecanism automat de transmitere periodică a datelor la fiecare 30 de secunde. Totodată, aplicația afișează starea curentă a comunicației, oferind informații despre succesul transmiterii sau eventualele erori apărute în timpul funcționării. Această abordare a permis realizarea unei componente stabile și ușor de extins ulterior.

Serverul central reprezintă componenta principală a sistemului și a fost implementat utilizând framework-ul Spring Boot. Acesta oferă o arhitectură modernă și modulară, bazată pe controllere REST și servicii separate pentru fiecare funcționalitate importantă a aplicației. Prin intermediul API-urilor REST dezvoltate, serverul primește datele de localizare de la aplicația Android, le procesează și le pune la dispoziția aplicației client implementate în Java Swing.

În cadrul serverului au fost implementate mai multe servicii importante pentru analiza datelor de localizare. Printre acestea se numără calculul vitezei medii de deplasare pe întreaga perioadă de transmitere, calculul distanței totale parcurse utilizând formula Haversine pentru determinarea distanței dintre două puncte GPS consecutive, precum și identificarea poziției exacte a vehiculului la un anumit moment de timp pe baza timestamp-ului asociat fiecărei transmisii. Aceste funcționalități pot avea aplicații practice în monitorizarea traseelor, analiza stilului de conducere sau verificarea poziției vehiculului în cazul unor incidente.

Un aspect important al proiectului este reprezentat de mecanismul de reconstrucție a traseului parcurs. Utilizând informațiile stocate în baza de date și serviciul OpenStreetMap, sistemul poate reconstitui traseele efectuate de vehicul și le poate afișa grafic în interfața utilizatorului. Pentru evitarea unirii incorecte a traseelor aparținând unor sesiuni diferite de deplasare, a fost introdus conceptul de „session_id”. Acesta permite separarea automată a traseelor atunci când între două puncte consecutive există o diferență mare de timp, corespunzătoare unor intervale în care aplicația sau serverul nu au fost active. Astfel, traseele sunt reconstruite într-un mod realist și corect din punct de vedere logic.

Baza de date utilizată în cadrul proiectului stochează toate informațiile necesare funcționării aplicației, incluzând coordonatele GPS, viteza, timestamp-ul și identificatorul sesiunii de deplasare. Comunicarea dintre server și baza de date este realizată utilizând Spring Data și Hibernate, ceea ce a permis dezvoltarea unei structuri clare și eficiente pentru accesul la date. Utilizarea Hibernate a oferit posibilitatea actualizării automate a structurii bazei de date și simplificarea operațiilor de persistare a informațiilor, reducând complexitatea implementării și timpul necesar dezvoltării.

Un element important implementat în cadrul proiectului îl reprezintă utilizarea platformei Tailscale pentru realizarea unei conexiuni securizate între aplicația Android și serverul central. Prin utilizarea unui VPN bazat pe WireGuard, telefonul și serverul au fost conectate într-o subrețea virtuală privată, permițând transmiterea datelor GPS din orice locație, fără a fi necesară conectarea la aceeași rețea locală. Această soluție a permis testarea aplicației într-un mod apropiat de utilizarea reală și a eliminat limitările impuse de funcționarea exclusivă într-o rețea Wi-Fi locală. Totodată, a fost necesară configurarea firewall-ului sistemului de operare pentru permiterea conexiunilor provenite din rețeaua Tailscale către serverul Spring Boot.

Interfața utilizatorului a fost realizată utilizând Java Swing și are rolul de a centraliza toate informațiile oferite de server într-o manieră simplă și intuitivă. Prin intermediul acesteia pot fi vizualizate statisticile principale ale sistemului, poziția curentă a vehiculului, ultimele coordonate transmise, traseul reconstruit pe hartă și informațiile asociate anumitor momente de timp. De asemenea, interfața oferă posibilitatea afișării vitezei medii și a distanței totale parcurse, precum și alte statistici relevante pentru analiza deplasării vehiculului.

Pe parcursul dezvoltării proiectului au fost întâlnite diverse probleme specifice aplicațiilor distribuite și comunicației prin Internet. Printre acestea se numără gestionarea permisiunilor Android pentru accesul la locație, configurarea comunicației HTTP între aplicație și server, problemele legate de firewall și accesul prin rețea, precum și sincronizarea și organizarea corectă a datelor GPS. Rezolvarea acestor probleme a contribuit semnificativ la înțelegerea modului de funcționare a aplicațiilor moderne bazate pe servicii web și arhitecturi distribuite.

În ceea ce privește contribuțiile personale, proiectul a implicat dezvoltarea integrală a arhitecturii software, proiectarea structurii aplicațiilor și implementarea principalelor funcționalități necesare sistemului. Au fost realizate atât partea de achiziție și transmitere a datelor GPS, cât și serverul central, baza de date și interfața de vizualizare. Totodată, a fost proiectată logica necesară pentru calculul statisticilor și reconstrucția traseelor, precum și mecanismul de separare a sesiunilor de deplasare.

Proiectul oferă multiple posibilități de dezvoltare ulterioară, atât din punct de vedere software, cât și hardware. O direcție importantă de extindere constă în înlocuirea telefonului Android utilizat în etapa actuală cu un modul dedicat de comunicație și localizare GPS/GSM, capabil să funcționeze independent și să fie integrat direct în autoturism. O astfel de implementare ar permite realizarea unui sistem mult mai compact și eficient energetic, eliminând dependența de un telefon mobil și oferind o soluție mai apropiată de sistemele profesionale utilizate în domeniul monitorizării flotelor auto. În plus, utilizarea unor module dedicate ar permite extinderea sistemului pentru monitorizarea simultană a mai multor vehicule, fiecare având propriul identificator și propriile statistici asociate.

Totodată, infrastructura software poate fi îmbunătățită prin migrarea serverului central către o platformă cloud sau către un server public dedicat. În etapa actuală, aplicația utilizează Tailscale pentru realizarea unei conexiuni securizate între dispozitive, însă într-o implementare reală la scară largă această soluție poate fi înlocuită cu servicii publice de hosting și API-uri securizate prin autentificare și criptare HTTPS. O astfel de arhitectură ar permite accesul simultan al mai multor utilizatori și ar facilita scalarea aplicației pentru un număr mare de vehicule conectate. De asemenea, mutarea serverului într-o infrastructură cloud ar permite implementarea unor mecanisme suplimentare de redundanță, backup automat și disponibilitate ridicată, aspecte importante pentru aplicațiile moderne bazate pe servicii web.

Interfața utilizatorului poate fi extinsă considerabil prin adăugarea unor funcționalități avansate de analiză și vizualizare a datelor. În forma actuală, aplicația oferă informații privind traseul, viteza medie, distanța totală și poziția la anumite momente de timp, însă pe viitor aceasta poate include statistici mai complexe legate de stilul de conducere, estimarea consumului de combustibil sau analiza eficienței deplasărilor. Utilizând datele GPS și viteza de deplasare, sistemul ar putea estima consumul aproximativ al autoturismului și costurile asociate

anumitor trasee, oferind utilizatorului informații suplimentare utile în utilizarea de zi cu zi.

O altă direcție importantă de dezvoltare o reprezintă analiza comparativă între mai multe vehicule aflate simultan în sistem. Prin extinderea arhitecturii actuale, aplicația ar putea identifica în timp real apropierea dintre două vehicule, posibilitatea întâlnirii acestora pe traseu sau chiar estimarea timpului până la intersecția unor rute. Astfel de funcționalități pot avea aplicații practice în domeniul logisticii, managementului flotelor sau organizării transportului în cadrul unor companii. În plus, prin utilizarea unor algoritmi de analiză predictivă, sistemul ar putea oferi sugestii privind traseele optime sau estimări privind timpul de sosire.

Aplicația poate fi extinsă și către funcționalități utile pentru utilizarea cotidiană a unui autoturism. Un exemplu îl reprezintă memorarea exactă a poziției unui vehicul într-o parcare de mari dimensiuni. În acest scenariu, sistemul ar putea salva automat ultima poziție cunoscută și ar putea ajuta utilizatorul să identifice rapid locul unde a fost parcat autoturismul. Această funcționalitate poate deveni foarte utilă în parcuri aglomerate sau în spații necunoscute utilizatorului.

Pe lângă extinderile funcționale, proiectul poate fi dezvoltat și din punct de vedere al interfeței grafice. Interfața Java Swing utilizată în etapa actuală a permis realizarea rapidă a unei aplicații desktop stabile și ușor de implementat, însă aceasta poate fi înlocuită în viitor cu o aplicație web modernă sau cu o interfață dezvoltată utilizând framework-uri grafice mai avansate. Astfel, utilizatorii ar putea accesa sistemul direct din browser, fără instalarea unei aplicații dedicate, iar afișarea datelor pe hartă ar putea beneficia de funcționalități interactive suplimentare și de o experiență vizuală îmbunătățită.

Realizarea acestui proiect a contribuit semnificativ la înțelegerea conceptelor studiate și a permis aplicarea practică a acestora într-un sistem complet funcțional. Pe parcursul implementării au fost utilizate numeroase tehnologii moderne, precum Spring Boot, REST API, Hibernate, Spring Data, MySQL, Android Services și OpenStreetMap, toate integrate într-o arhitectură distribuită coerentă. Dezvoltarea proiectului a implicat atât partea de comunicație între aplicații, cât și gestionarea bazelor de date, proiectarea interfețelor grafice și implementarea serviciilor web.

Procesul de dezvoltare a evidențiat și dificultățile specifice aplicațiilor distribuite și sistemelor bazate pe comunicație în timp real. Au fost necesare configurări de rețea, gestionarea accesului prin firewall, implementarea comunicației dintre dispozitive aflate în rețele diferite și rezolvarea problemelor legate de sincronizarea datelor. În același timp, aceste provocări au oferit o experiență practică importantă privind modul de funcționare al sistemelor moderne de comunicații și al aplicațiilor client-server.

Din punct de vedere personal și educațional, proiectul a reprezentat o oportunitate de aprofundare a dezvoltării aplicațiilor Java și a integrării tehnologiilor software cu servicii bazate pe localizare. Implementarea sistemului a permis înțelegerea modului în care datele GPS

pot fi colectate, procesate și utilizate pentru realizarea unor aplicații reale, cu utilitate practică în domeniul transportului și al monitorizării vehiculelor. Totodată, proiectul a demonstrat importanța unei arhitecturi software bine organizate și avantajele utilizării unei abordări modulare în dezvoltarea aplicațiilor complexe.

În concluzie, sistemul realizat demonstrează funcționarea practică a unei aplicații moderne de monitorizare GPS și reprezintă o bază solidă pentru dezvoltări viitoare. Arhitectura modulară implementată permite extinderea facilă a funcționalităților și adaptarea aplicației pentru scenarii reale de utilizare. Prin combinarea tehnologiilor software moderne cu serviciile de localizare și comunicație în Internet, proiectul evidențiază modul în care pot fi dezvoltate soluții scalabile și utile pentru managementul vehiculelor și analiza deplasărilor în timp real.